

Table 1: Hyperparameters and Configuration Details by Architecture

Architecture	Component	Hyperparameters & Settings
Base KAGNN	GAT Backbone	Layers: 5 Hidden Dimension: 256 Attention Heads: 4
	Input Dims	Atom Features: 133-dim, Bond Features: 14-dim Fingerprints: 1024-bit ECFP
	Optimization	Optimizer: AdamW Learning Rate: 3e-4, Weight Decay: 1e-5 Loss: Smooth L1 Early Stopping: 20 epochs
	Preprocessing	Target (RT) Scaling: RobustScaler
Forward Residual (KA-GNN)	Stage 1 (KAGNN)	Configuration identical to Base KAGNN Backbone Output: Initial RT + 256-dim embeddings
	Stage 2 Input	288-dim vector (256-dim embeddings + 32-dim molecular descriptors) Scaling: StandardScaler XGBoost:
	Stage 2 (PGM)	n_estimators: 50, max_depth: 6, learning_rate: 0.05 Bayesian Ridge: max_iter: 500 Optimization: 30 Optuna trials (targeting MAE)
Reverse Residual (KA-GNN)	Stage 1 (PGM)	Inputs: 32 descriptors + 1024-bit ECFP4 <i>Trained on residuals from Stage 1</i>
	Stage 2 (KAGNN)	Layers: 5 (GAT fallback) Hidden Dimension: 256, Attention Heads: 4 Dropout: 0.1 Gradient Clipping: Enabled
	Optimization	Loss: Smooth L1, Optimizer: AdamW
GCN-based GNN Hybrid (Exp. 3)	MolGCN Backbone	Graph stream: $3 \times \text{GCNConv}(256) \rightarrow \text{global mean pool}$ FP stream: $\text{LayerNorm}(1024) \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow 256\text{-dim}$ Fusion: $\text{concat}(512) \rightarrow \text{Linear}(256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(1)$
	Node Features	23-dim (atomic type, hybridisation, aromaticity, charge, implicit-H, ring, chirality)
	PGM Component	BayesianRidge on ECFP4 + 32 descriptors (1056-dim); forward: corrects GNN residuals; reverse: provides baseline
	Optimization	AdamW, LR 3×10^{-4} , weight decay 10^{-5} , Huber loss ($\delta = 1.0$), early stopping patience 20
	Training Time	<10 min (vs. ~ 193 min for KA-GNN)

RT Prediction using KAGNN

Algorithm 1 RT Prediction using KAGNN

```
1: Input: SMRT dataset (CSV, ECFP, SDF), Hyperparameters
2: Output: Predicted RT values  $\hat{y}$ 
3:
4: // Data Loading
5: Load CSV file and extract PubChem IDs and RT values
6: Load ECFP fingerprints from text file
7: Load molecular structures from SDF file
8: Filter dataset to common IDs across all three sources
9: Normalize RT values using RobustScaler
10: Split data into train/validation/test sets
11:
12: // Model Architecture
13: Initialize KAGNN parameters:  $W^{(k)}, b^{(k)}, \mathbf{w}, \mathbf{b}$ 
14:
15: // Training
16: for each epoch do
17:   for each batch in training data do
18:     Extract molecular graph  $G$  from molecule
19:     Extract ECFP fingerprint  $\mathbf{f}$ 
20:     Extract graph features  $\mathbf{h}_G$  using GNN layers
21:     Extract fingerprint features  $\mathbf{h}_E$  using MLP
22:     Concatenate:  $\mathbf{h} = [\mathbf{h}_G || \mathbf{h}_E]$ 
23:     Compute predicted RT:  $\hat{y} = \text{Fusion}(\mathbf{h})$ 
24:     Compute loss using SmoothL1Loss
25:     Backpropagate and update parameters
26:   end for
27:   Evaluate on validation set
28:   if validation loss improved then
29:     Save best model
30:   else
31:     Increment early stopping counter
32:   end if
33: end for
34:
35: Load best model and evaluate on test set
36: return  $\hat{y}$ 
```

Two-Stage RT Prediction: KAGNN followed by PGM Residual Correction

Algorithm 2 Two-Stage RT Prediction: KAGNN followed by PGM Residual Correction

```
1: Input: SMRT dataset (CSV, ECFP, SDF), Hyperparameters
2: Output: Predicted RT values  $\hat{y}$ 
3:
4: // Data Loading
5: Load CSV file and extract PubChem IDs and RT values
6: Load ECFP fingerprints from text file
7: Load molecular structures from SDF file
8: Extract molecular descriptors from molecules
9: Filter dataset to common IDs and normalize RT values
10: Split data into train/validation/test sets
11:
12: // Stage 1: Train KAGNN Backbone
13: Initialize KAGNN parameters:  $W^{(k)}, b^{(k)}, \mathbf{w}, \mathbf{b}$ 
14: for each epoch do
15:   for each batch in training data do
16:     Extract molecular graph  $G$  and ECFP  $\mathbf{f}$ 
17:     Extract graph features  $\mathbf{h}_G$  using GNN layers
18:     Extract fingerprint features  $\mathbf{h}_E$  using MLP
19:     Concatenate:  $\mathbf{h} = [\mathbf{h}_G || \mathbf{h}_E]$ 
20:     Compute predicted RT:  $\hat{y}_{\text{KAGNN}} = \text{Predictor}(\mathbf{h})$ 
21:     Compute loss using SmoothL1Loss
22:     Backpropagate and update KAGNN parameters
23:   end for
24:   Evaluate on validation set
25:   if validation loss improved then
26:     Save best model
27:   else
28:     Increment early stopping counter
29:   end if
30: end for
```

Algorithm 2 Two-Stage RT Prediction: KAGNN followed by PGM Residual Correction (continued)

31: // **Stage 2: Train PGM on Residuals**

32: Load best KAGNN model

33: **for** each sample in training set **do**

34: Extract graph G , ECFP $\mathbf{f}$, and true RT y

35: Compute KAGNN prediction and embeddings: $\hat{y}_{\text{KAGNN}}, \mathbf{h}$

36: Calculate residual: $r = y - \hat{y}_{\text{KAGNN}}$

37: Combine features: $\mathbf{X} = [\mathbf{h} || \mathbf{d}_{\text{mol}}]$ // 256-dim KA-GNN embeddings + 32 RDKit descriptors = 288-dim

38: **end for**

39: Train XGBoost regressor on $\mathbf{X}, r$

40: Train Bayesian Ridge regressor on $\mathbf{X}, r$

41: Compute PGM residual predictions:

$$\hat{r}_{\text{PGM}} = \frac{\hat{r}_{\text{XGB}} + \hat{r}_{\text{BR}}}{2}$$

42: // **Final Prediction**

43: **for** each sample in test set **do**

44: Compute KAGNN base prediction: $\hat{y}_{\text{KAGNN}}$

45: Extract combined features $\mathbf{X}$

46: Compute PGM residual correction: $\hat{r}_{\text{PGM}}$

47: Final prediction: $\hat{y}_{\text{final}} = \hat{y}_{\text{KAGNN}} + \hat{r}_{\text{PGM}}$

48: **end for**

49: **return** $\hat{y}_{\text{final}}$

Two-Stage RT Prediction: PGM followed by KAGNN Residual Correction

Algorithm 3 Two-Stage RT Prediction: PGM followed by KAGNN Residual Correction

```
1: Input: SMRT dataset (CSV, ECFP, SDF), Hyperparameters
2: Output: Predicted RT values  $\hat{y}$ 
3:
4: // Data Loading
5: Load CSV file and extract PubChem IDs and RT values
6: Load ECFP fingerprints from text file
7: Load molecular structures from SDF file
8: Extract molecular descriptors from molecules
9: Filter dataset to common IDs and normalize RT values
10: Split data into train/validation/test sets
11:
12: // Stage 1: Train PGM Ensemble
13: Combine ECFP fingerprints with molecular descriptors:
14:  $\mathbf{X} = [\mathbf{f}_{\text{ECFP}} || \mathbf{d}_{\text{mol}}]$ 
15: Train XGBoost regressor on  $\mathbf{X}, y_{\text{RT}}$ 
16: Train Bayesian Ridge regressor on  $\mathbf{X}, y_{\text{RT}}$ 
17: Compute PGM predictions:
18:  $\hat{y}_{\text{PGM}} = \frac{\hat{y}_{\text{XGB}} + \hat{y}_{\text{BR}}}{2}$ 
19:
20: // Stage 2: Train KAGNN on Residuals
21: Initialize KAGNN parameters:  $W^{(k)}, b^{(k)}, \mathbf{w}, \mathbf{b}$ 
22: for each epoch do
23:   for each batch in training data do
24:     Extract molecular graph  $G$  and ECFP  $\mathbf{f}$ 
25:     Compute PGM prediction:  $\hat{y}_{\text{PGM}}$ 
26:     Calculate residual:  $r_{\text{true}} = y_{\text{RT}} - \hat{y}_{\text{PGM}}$ 
27:     Extract graph features  $\mathbf{h}_G$  using GNN layers
28:     Extract fingerprint features  $\mathbf{h}_E$  using MLP
29:     Concatenate features:  $\mathbf{h} = [\mathbf{h}_G || \mathbf{h}_E]$ 
30:     Predict residual:  $\hat{r} = \text{RefinementHead}(\mathbf{h})$ 
31:     Compute loss:  $\mathcal{L} = \text{SmoothL1Loss}(\hat{r}, r_{\text{true}})$ 
32:     Backpropagate and update KAGNN parameters
33:   end for
34:   Evaluate on validation set
35:   if validation loss improved then
36:     Save best model
37:   else
38:     Increment early stopping counter
39:   end if
40: end for
41:
42: // Final Prediction
43: Load best KAGNN model
44: for each sample in test set do
45:   Compute PGM base prediction:  $\hat{y}_{\text{PGM}}$ 
46:   Compute KAGNN residual correction:  $\hat{r}_{\text{KAGNN}}$ 
47:   Final prediction:  $\hat{y}_{\text{final}} = \hat{y}_{\text{PGM}} + \hat{r}_{\text{KAGNN}}$ 
48: end for
49: return  $\hat{y}_{\text{final}}$ 
```

Figures of the Experiment 1’s results

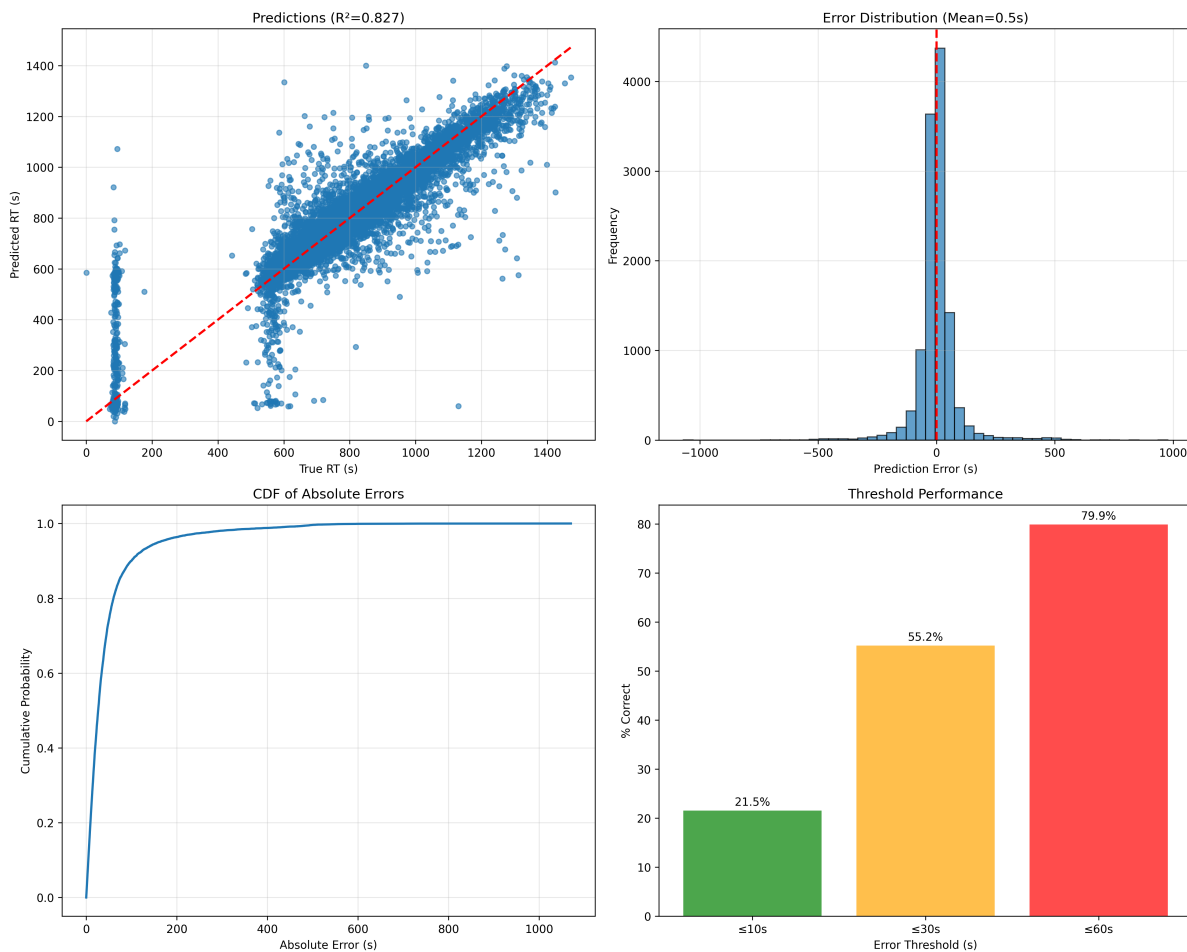


Figure 1: Performance characteristics of the standalone KA-GNN model on the SMRT test set ($n = 11,994$ compounds). The scatter plot demonstrates strong linear correlation between predicted and experimental retention times (Pearson $r = 0.9068$, Spearman $\rho = 0.9283$), while the coefficient of determination $R^2 = 0.8190$ indicates that the model explains over 81% of variance in retention behavior. The error distribution exhibits high kurtosis, reflecting the challenge of predicting outlier compounds with unusual chromatographic behavior. Statistical validation confirms minimal systematic bias (mean error = 2.49 s, $p = 0.307$).

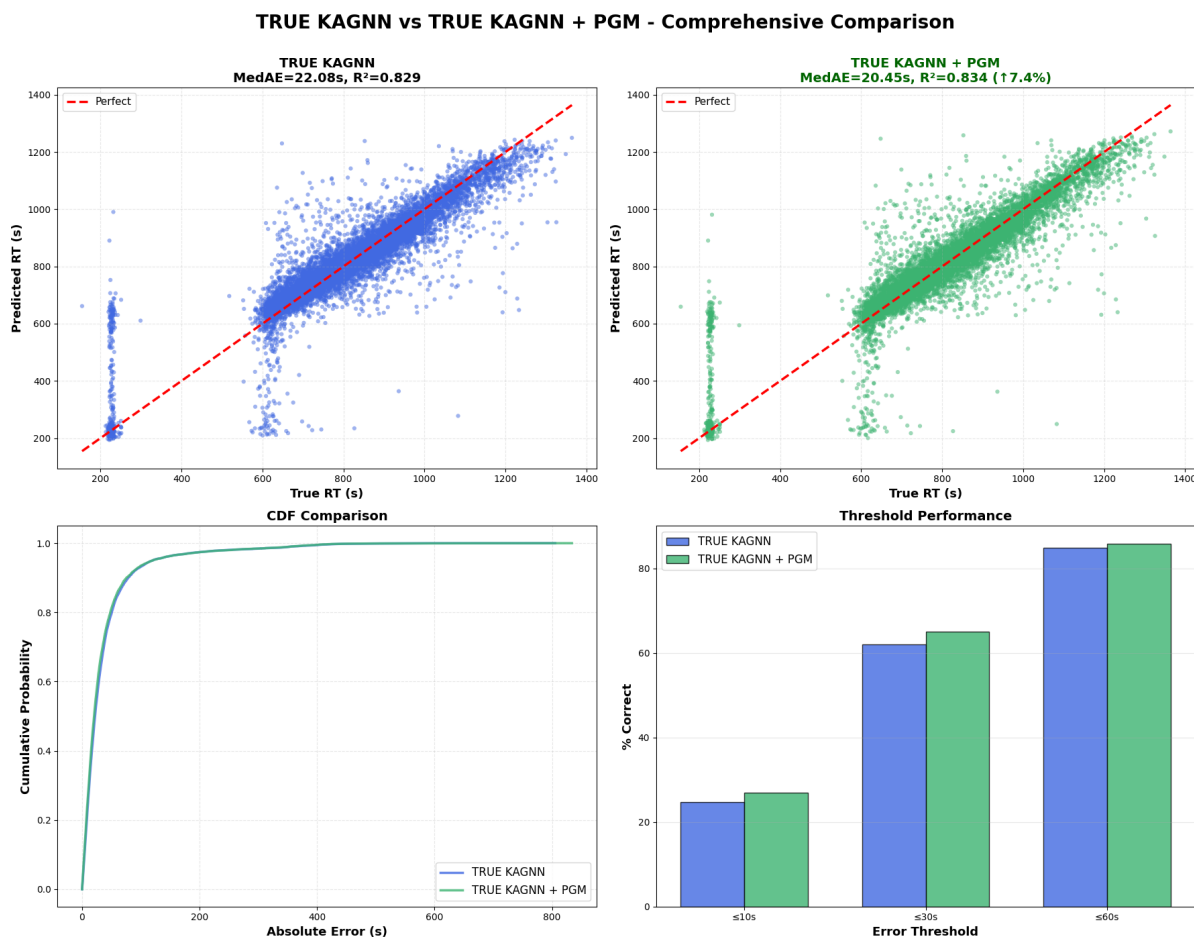


Figure 2: Comprehensive performance comparison between the forward hybrid (KA-GNN +PGM) and standalone KA-GNN models. The hybrid architecture achieves MedAE = 20.56 s and $R^2 = 0.8239$, representing a 23.2% improvement in median absolute error over the standalone model (MedAE = 26.76 s, $R^2 = 0.8190$). Scatter plots demonstrate strong linear correlation with experimental retention times for both models, while threshold performance comparisons show substantial gains in the ± 10 s (26.24% vs 20.74%), ± 30 s (63.82% vs 54.47%), and ± 60 s (85.68% vs 79.71%) windows. The improvement in high-precision predictions is particularly notable, with approximately 5.5% more compounds predicted within the stringent ± 10 second window.

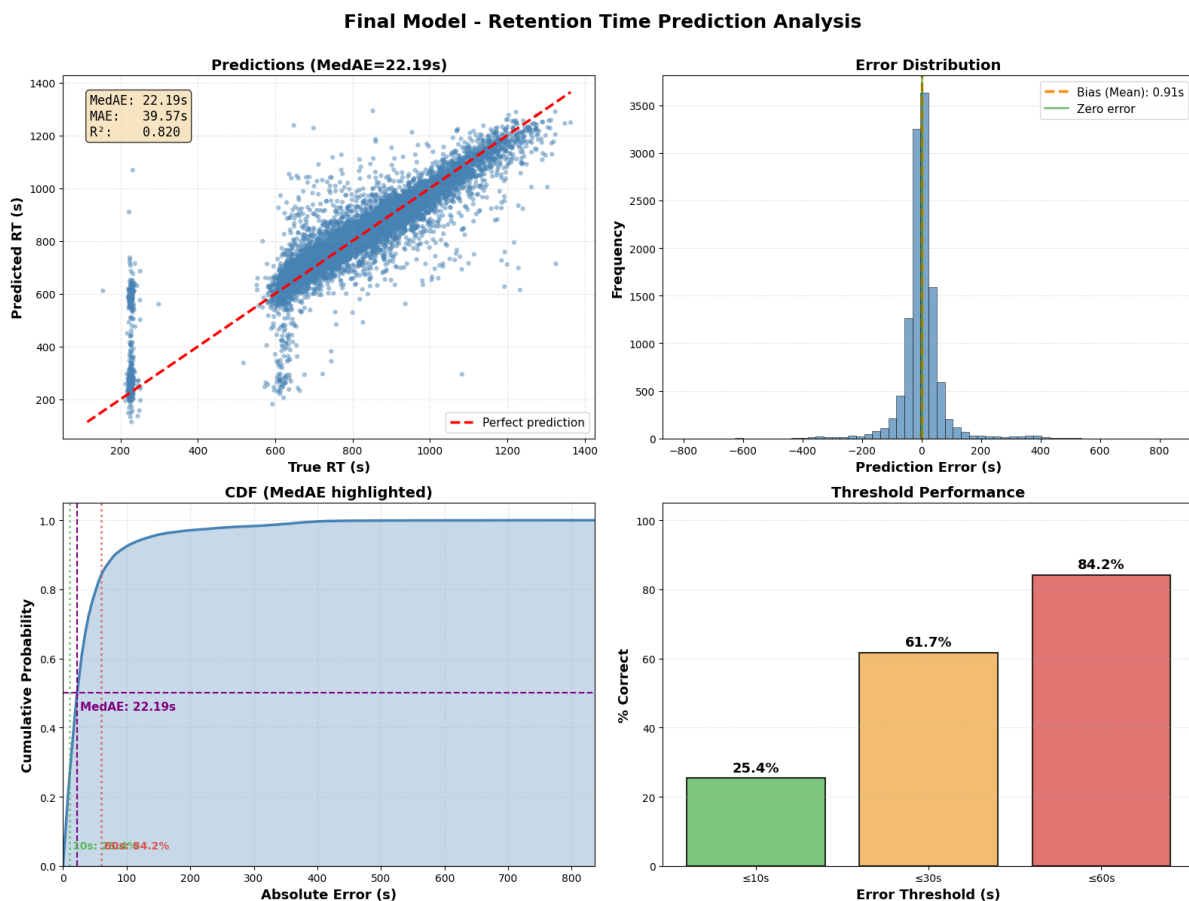


Figure 3: Comprehensive performance analysis of the reverse hybrid (PGM +KA-GNN) model on the SMRT test set. The model achieves MedAE = 22.19 s with median relative error of 2.84%, representing a 38.61% improvement over the PGM baseline. Strong correlation metrics (Pearson $r = 0.9061$, Spearman $\rho = 0.9257$) and high coefficient of determination ($R^2 = 0.8202$) confirm robust performance. The model exhibits minimal systematic bias (mean error = 1.18 s) and delivers excellent threshold coverage with 25.40%, 61.70%, and 84.18% of predictions falling within ± 10 s, ± 30 s, and ± 60 s windows, respectively. The scatter plot demonstrates tight clustering along the diagonal, indicating strong agreement between predicted and experimental values across the full retention time range.

Figures of the Experiment 2's results

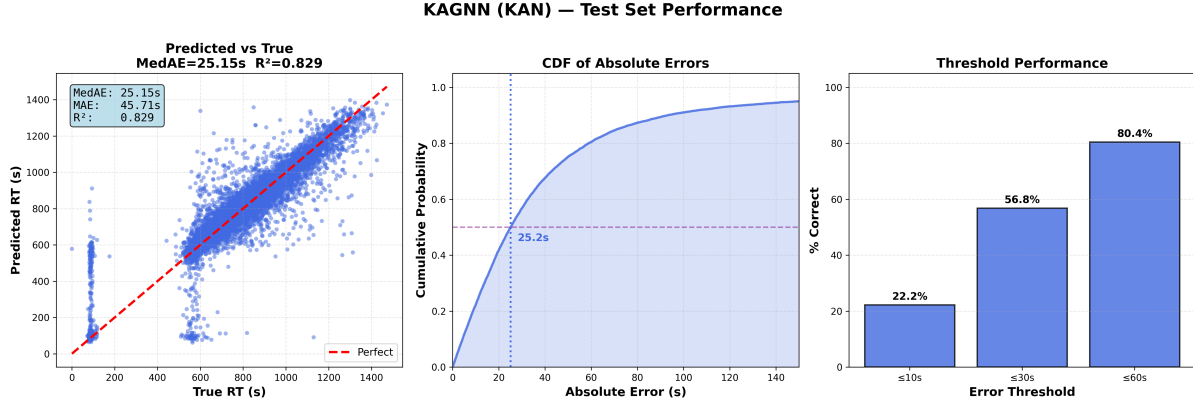


Figure 4: Test-set performance of the standalone KA-GNN under Experiment 2 class-wise training (one representative fold). MedAE = 25.15 s, $R^2 = 0.829$, MAE = 45.71 s. Threshold coverage: 22.2% (± 10 s), 56.8% (± 30 s), 80.4% (± 60 s). Results are consistent with the Experiment 2 mean of 25.93 ± 0.44 s reported in Table 3.

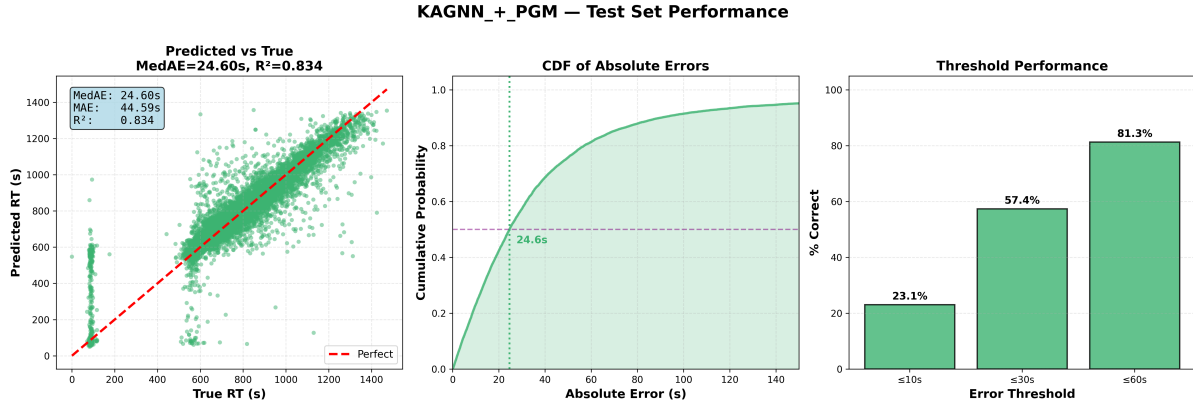


Figure 5: Test-set performance of the Forward Hybrid (KA-GNN $\rightarrow$ PGM) under Experiment 2 class-wise training ($n = 11,994$). MedAE = 24.60 s, $R^2 = 0.8340$, MAE = 44.59 s, RMSE = 84.11 s, Pearson $r = 0.9134$, Spearman $\rho = 0.9330$. Threshold coverage: 23.07% (± 10 s), 57.37% (± 30 s), 81.35% (± 60 s). Statistical significance vs. standalone KA-GNN: $t = 9.67$, $p = 4.76 \times 10^{-22}$; Wilcoxon $W = 31,965,816$, $p = 5.01 \times 10^{-26}$.

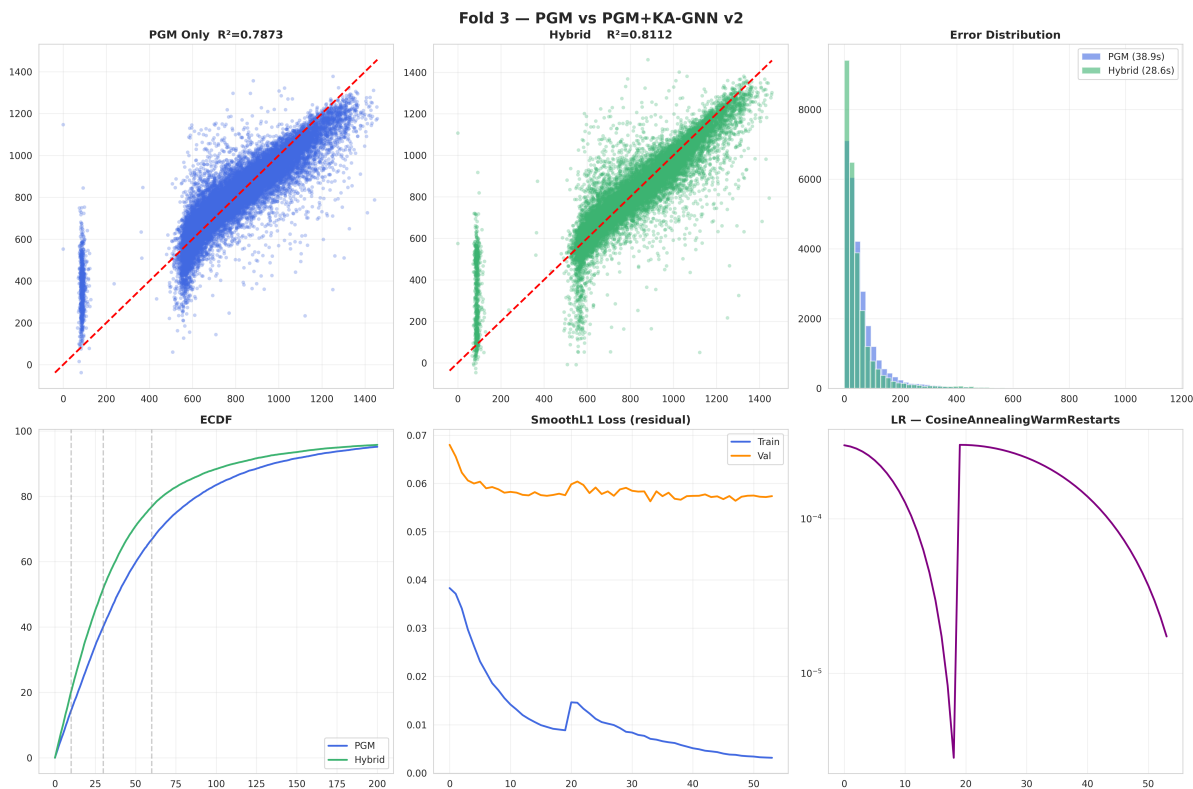


Figure 6: Fold 3 evaluation of the Reverse Hybrid (PGM $\rightarrow$ KA-GNN). Top left: PGM-only scatter ($R^2 = 0.787$, MedAE = 38.95 s). Top centre: Full hybrid scatter ($R^2 = 0.811$, MedAE = 28.60 s). Top right: Overlaid error distributions confirm the hybrid shift relative to PGM-only. Bottom left: ECDF comparison. Bottom centre: SmoothL1 residual loss (train/val), early stopping at epoch 54 (best val = 0.05629), no overfitting. Bottom right: CosineAnnealingWarmRestarts LR schedule ($T_0 = 20$). Statistical significance: $t = 42.73$, $p \approx 0$; Wilcoxon $W = 114,861,441$, $p \approx 0$.



Figure 7: Class-wise MedAE of the standalone KA-GNN (Experiment 2, aggregated over 3 folds). Minority classes including Lipids and Carbohydrates exhibit substantially higher errors, motivating the hybrid correction strategies evaluated in subsequent architectures.

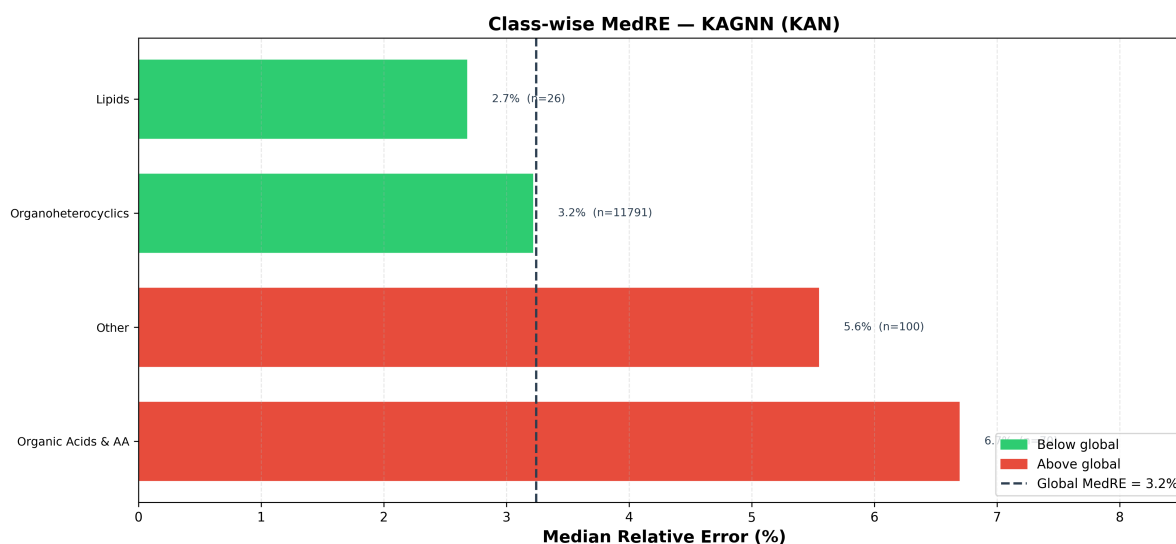


Figure 8: Class-wise MedRE of the standalone KA-GNN (Experiment 2). Proportional prediction difficulty scales with chemical class complexity and minority status, independently of absolute retention time magnitude.

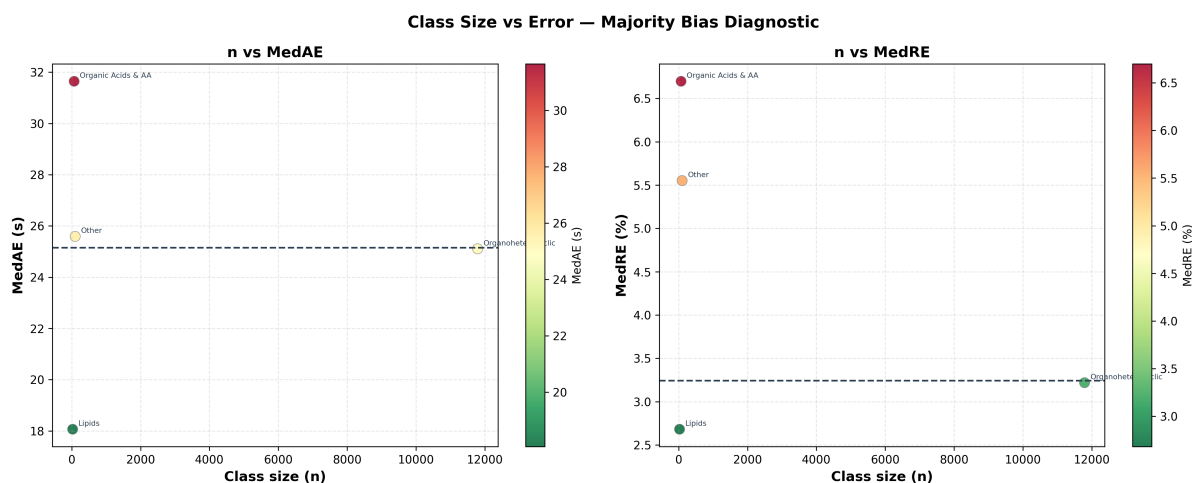


Figure 9: Class size versus MedAE for the standalone KA-GNN. A clear negative correlation confirms that minority classes with fewer than 5,000 compounds exhibit substantially higher MedAE, motivating class-stratified augmentation strategies.

Class-wise Analysis of Experiment 2

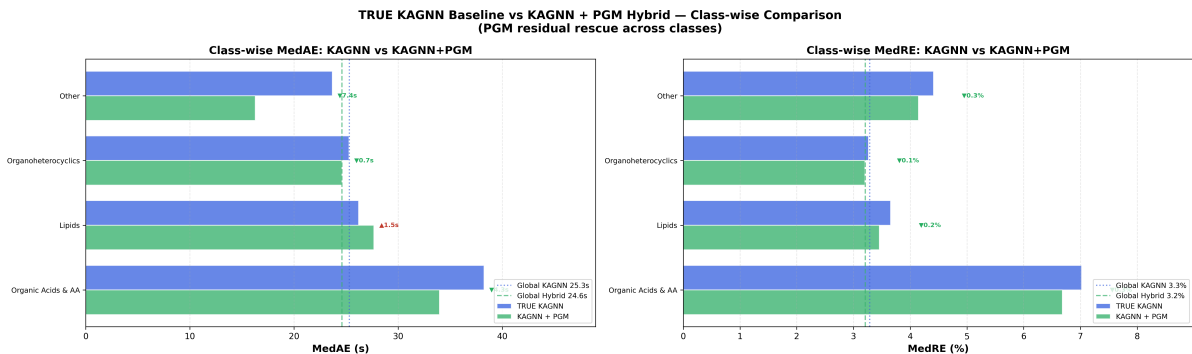


Figure 10: Class-wise MedAE comparison between the standalone KA-GNN and the Forward Hybrid (KA-GNN $\rightarrow$ PGM) across chemical classes (Experiment 2). The Forward Hybrid improves dominant classes but selectively degrades Lipids (+9.7 s) and Organic Acids (+1.9 s), revealing a class-differential failure mode induced by the PGM residual correction stage.

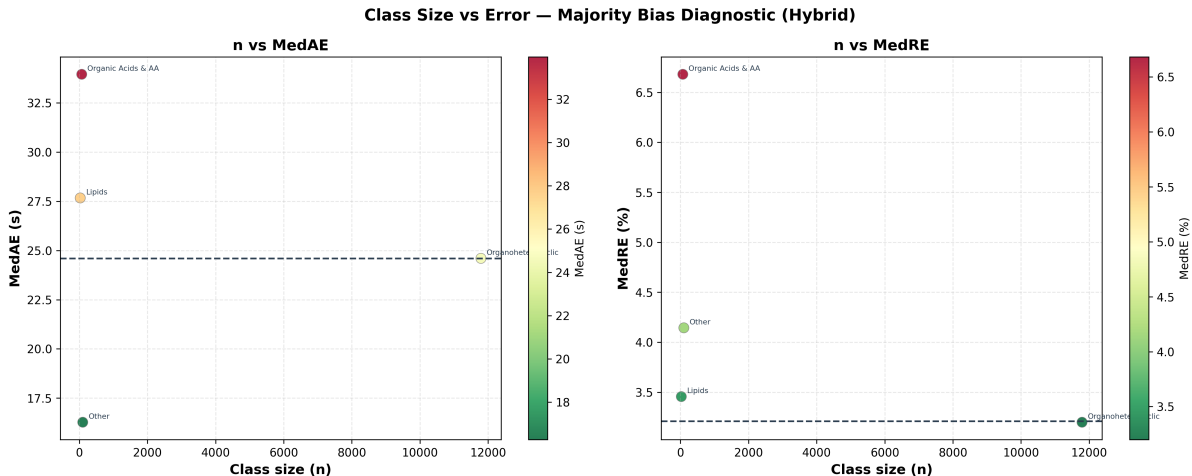


Figure 11: Class size versus MedAE for the Forward Hybrid. Despite sufficient training samples, the PGM residual correction over-corrects for minority classes whose descriptor-based features diverge from the residual structure learned from the majority.

1 Introduction

Retention time (RT) prediction plays a critical role in liquid chromatography–mass spectrometry (LC–MS) workflows, aiding in the identification and characterization of small molecules in pharmaceutical and metabolomics studies [1, 2, 7]. Accurate RT prediction reduces false positives, improves metabolite annotation, and enhances the throughput of analytical pipelines [13].

Traditional approaches often rely on hand-crafted molecular descriptors or quantitative structure–retention relationships (QSRR), which can be limited in capturing complex structural effects on chromatographic behavior [10]. Recent advances have leveraged graph neural networks (GNNs) to directly model molecular structures as graphs, allowing for end-to-end learning of RT from atom and bond features. For example, GNN-RT [12], DeepGCN-RT [6], and RT-Transformer [11] demonstrate the potential of deep GNN architectures on the SMRT dataset, achieving median absolute errors on the order of 10–20 seconds. Many of these models further improve performance by fine-tuning on additional or domain-specific datasets, demonstrating the benefit of transfer learning strategies in RT prediction.

A common preprocessing strategy in these studies is the removal of non-retained compounds (typically molecules with $RT < 2$ min), which reduces noise and improves predictive accuracy. While effective, this limits the model’s ability to generalize to the full chemical space encountered in pharmaceutical LC–MS workflows.

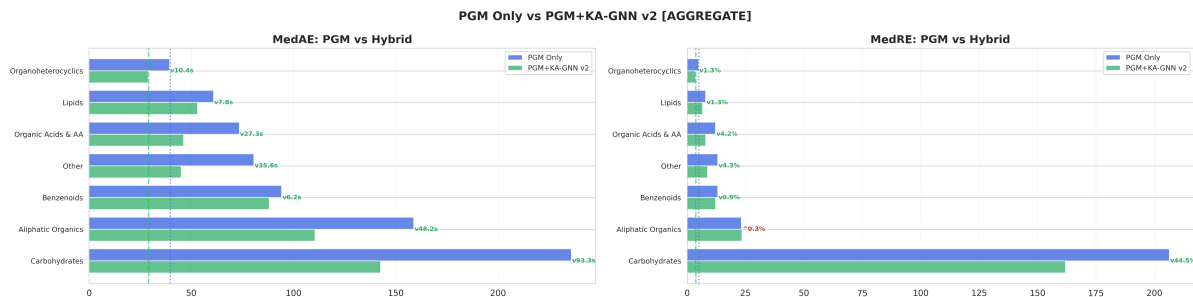


Figure 12: Class-wise MedAE of the Reverse Hybrid (PGM $\rightarrow$ KA-GNN) compared to the PGM baseline (Experiment 2, aggregated over 3 folds). Every class improves without exception; the largest gains are for Carbohydrates (-93.3 s), Aliphatic Organics (-48.2 s), and Other (-35.6 s).

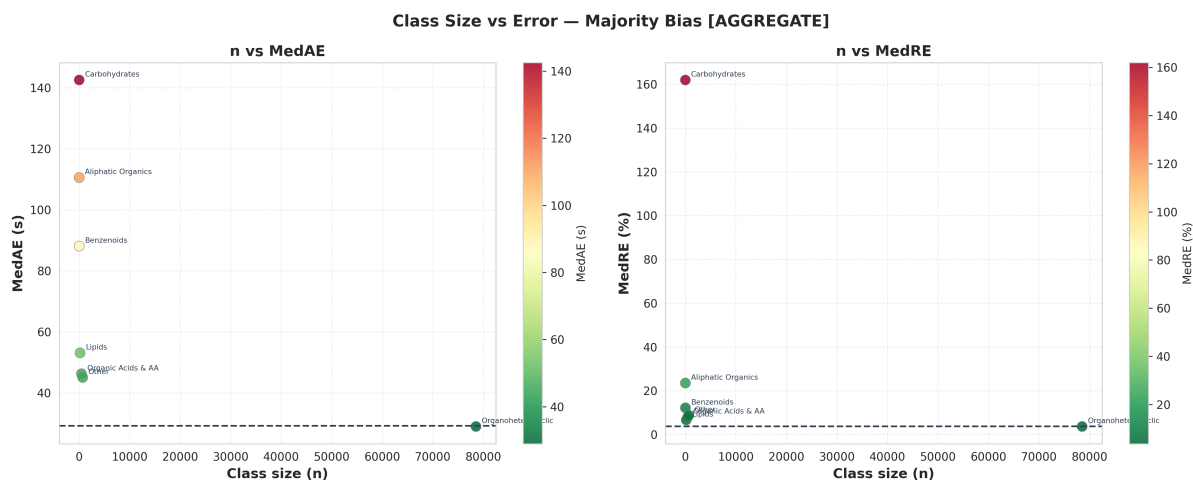


Figure 13: Class size versus MedAE for the Reverse Hybrid. Unlike the Forward Hybrid, no minority class is degraded relative to the PGM baseline, confirming that coarse-to-fine ordering is the robust choice for chemically diverse deployment scenarios.